

Self-Hosted Geocoding for Maine

Pelias and a PostgreSQL/PostGIS geocoder (pgeo): design, tuning, accuracy and capacity

pelias_maine project

Contents

Executive summary	1
1. Introduction	2
2. Methods	4
3. Results	10
4. Discussion	23
5. Conclusions	24
6. Next steps	24
Appendix A. Reproducing this report	24
Appendix B. pgeo storage	24

Project version 0.11.0 (pgeo 0+unknown) · built 2026-09-19 from commit 68b6dfa (with uncommitted changes) · data snapshot 2026-09-19 · load-test runs 20260918-1834, 20260918-2051, 20260919-0718, 20260918-2343-pgeo, 20260919-0350-pgeo. Generated by scripts/build_report.sh; every number, table and chart in this report is computed from the saved results listed in report/inputs.toml.

Executive summary

This study built and compared two self-hosted geocoders for the State of Maine on the same open data: **Pelias**, the established open-source geocoder (Elasticsearch plus five Node.js and C++ services), and **pgeo**, a geocoder written for this project that runs entirely inside PostgreSQL 18 with PostGIS, served either by a thin FastAPI application or, with no application code at all, by PostgREST (“pure SQL”). Pelias served as the reference and as the test oracle for pgeo’s API.

Accuracy. On 1,560 engine-independent test cases whose answers come from the source data, pgeo answers **95.8%** correctly against **75.5%** for Pelias. The gap is largest where people make mistakes: queries with typos (87% vs 30%), places that do not exist (95% vs 41% handled correctly), lakes and summits (95% vs 41%) and venues (94% vs 68%). Under progressive corruption of 300 base queries, pgeo still answers 75% with two character errors per query, where Pelias answers 10%.

Capacity. Pelias is the faster engine: at equal CPU it keeps 4 to 24 times more concurrent users within the latency targets. pgeo is the smaller one: it meets every target for the 3-user goal of this project in a 1.6 GB memory budget on one vCPU, where Pelias needs about 8.3 GB to run at all.

Compatibility and extensions. pgeo passes all 27 cases of a Pelias API compatibility contract on both front ends, so existing Pelias clients work unchanged, and adds a structured-address endpoint (USPS Publication 28) for the LANCER dispatch application.

Recommendation. For the stated need (a handful of users, a small VM or VPS, messy input, fewest components) pgeo in its pure-SQL form is the better fit; Pelias remains the choice for hundreds of concurrent users per vCPU. Section “Conclusions” maps scenarios to tools.

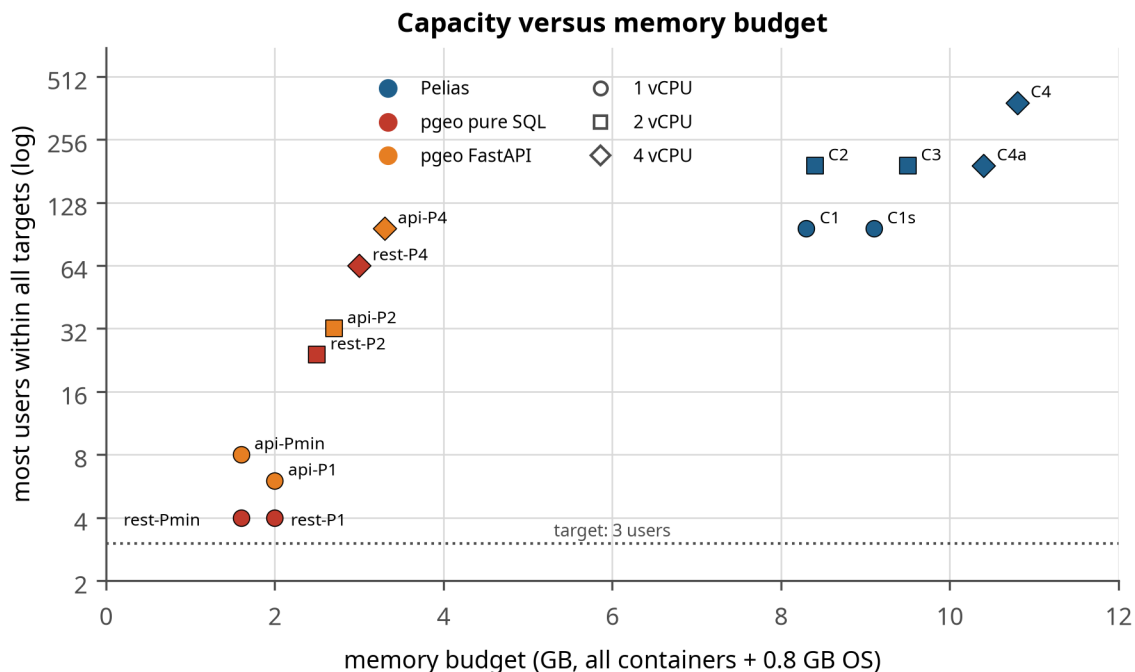


Figure 1. Capacity against memory budget for every tested configuration of both engines. Each point is one configuration; the vertical axis is the most concurrent users for which every endpoint stayed within its latency target (log scale). pgeo configurations sit at the left (1.6-3.3 GB), Pelias at the right (8.3-10.8 GB). The dotted line is the project's 3-user goal.

Table 1. Key findings.

Question	Answer	Evidence
Which engine is more accurate?	pgeo, by about 20 points overall and 50+ points on typos	Section 3.1
Which is faster under load?	Pelias, by a factor of 4 to 24 per CPU	Section 3.4
Smallest configuration for 3 users?	pgeo: 1 vCPU, 1.6 GB. Pelias: 1 vCPU, ~8.3 GB	Section 3.5
Can PostgreSQL alone be the API?	Yes: PostgREST in front of SQL functions, identical accuracy	Sections 2.1, 3.8
Does more data slow Pelias?	Addresses barely; OSM and Overture places cost one ramp step each	Section 3.6
Drop-in replacement for Pelias clients?	Yes for the documented API (27 of 27 contract cases)	Section 3.8

1. Introduction

1.1 Motivation

Geocoding (turning a typed address or place name into coordinates) and reverse geocoding (turning a map position into an address) are basic services for any application that deals with places. Commercial geocoding APIs charge per request, send every query to a third party, and change without notice. This project set out to host a feature-rich geocoder for Maine (and later New Hampshire) on the owner's own infrastructure: a Proxmox server on the home LAN, reachable over HTTPS, with a map-based demonstration page.

Two further needs shaped the work. First, the geocoder is meant to feed **LANCER**, a dispatch-and-billing application whose client sites carry sensitive, encrypted street addresses; queries that contain such addresses

should not leave the owner’s network or linger in logs. Second, a guiding principle (G6) asked for **the stack with the fewest additional components, keeping data in PostgreSQL, unless that produces an inferior or overly complex product**. That principle motivated building pgeo alongside Pelias and measuring both.

1.2 Starting conditions

Item	State at the start
Infrastructure	Proxmox VE 9.2 node (prox82); Debian 13 cloud-init template; a TLS-terminating Caddy on the “wharf” VM with public DNS-01 certificates; Forgejo for source control
Target host	VM 120 (pelias.wharf.fluidgrid.org), 4 vCPU and 10 GB RAM, NVMe storage, LAN only
Build and test host	Workstation: AMD Ryzen 5 3600 6-Core Processor, 12 threads, 31 GB RAM
Reference recipe	The Pelias docker project for Texas, adapted for Maine
Data	Open data only; no OpenAddresses API token (the statewide Maine file was used)
Goals	G1 accurate Maine geocoding; G2 LAN-only, HTTPS; G3 reproducible build; G4 demo page; G5 minimum resources for 3 users; G6 fewest components

1.3 Questions

1. How accurate is each engine on Maine data, including misspelled, variant and impossible queries?
2. What are the minimum server resources for 3 concurrent users, and where does each engine break when users are added?
3. How does adding data layers change performance?
4. Can PostgreSQL itself be the API endpoint (G6) without an inferior product?
5. Can pgeo replace Pelias for existing clients, and what does it add?

2. Methods

2.1 Systems built

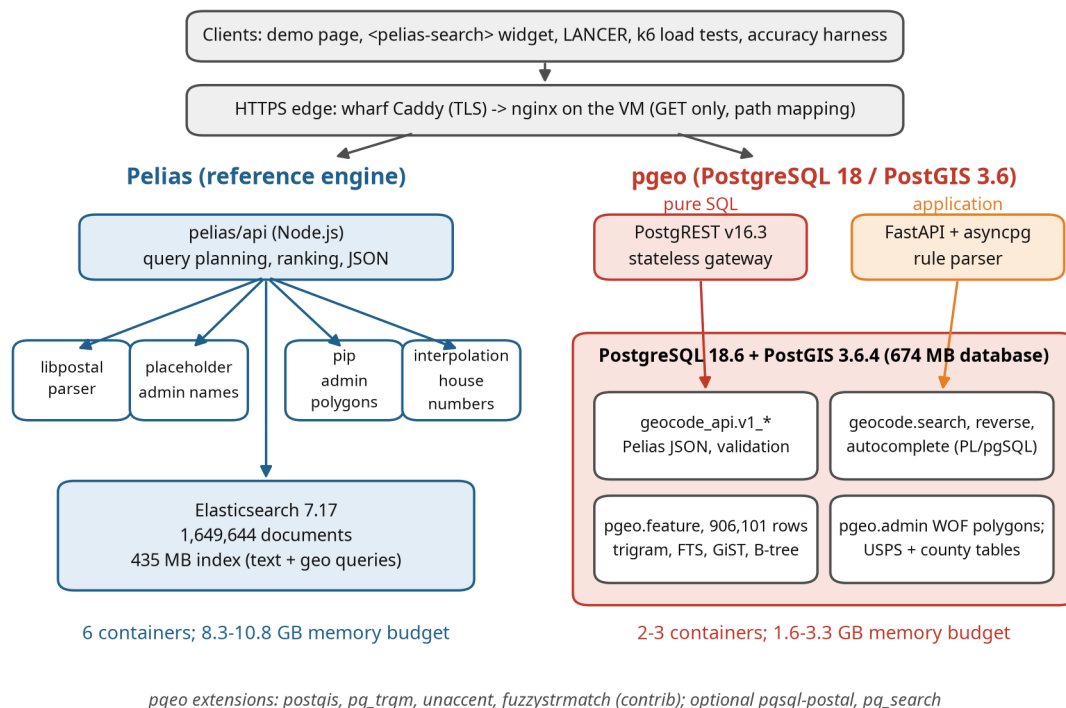


Figure 2. System architecture. Clients reach both engines through the same HTTPS edge. Pelias (left) is an API service coordinating four helper services and Elasticsearch. pgeo (right) keeps data, parsing, ranking and JSON formatting inside PostgreSQL; the HTTP front end is either PostgREST (a stateless gateway, no application code) or a thin FastAPI application.

Pelias was deployed from pinned images (API, libpostal parser, placeholder, point-in-polygon, interpolation, Elasticsearch 7.17.27) with the index built on the workstation and shipped to VM 120 as an Elasticsearch snapshot. Nginx on the VM serves the demo page and the API under strict security headers; the wharf Caddy terminates TLS.

pgeo runs on PostgreSQL 18.6 (Debian 18.6-1.pgdg13+2) with PostGIS 3.6.4 and only core and contrib extensions (pg_trgm for trigram similarity, unaccent, fuzzystmatch). Every query step is a SQL or PL/pgSQL function (Figure 3 shows the forward-search path). The public API is a schema of functions (geocode_api.v1_search, v1_reverse, ...) that return Pelias-shaped GeoJSON, so any HTTP layer that can call a function can serve it. Two front ends were built and measured:

- **PostgREST** (pure SQL): maps GET /rpc/v1_search?text=. . . to the function; an nginx edge maps Pelias paths (/v1/search) to RPC paths. Chosen over an in-database HTTP server (Omnigres) because it keeps third-party code out of the database process, and over OpenResty because it needs no custom code (docs/HTTP_OPTIONS.md).
- **FastAPI** (application): the same SQL functions behind a Python service, kept as the baseline to show what an application layer adds or costs.

Every step runs inside PostgreSQL; the FastAPI and PostgREST front ends pass parameters in and JSON out

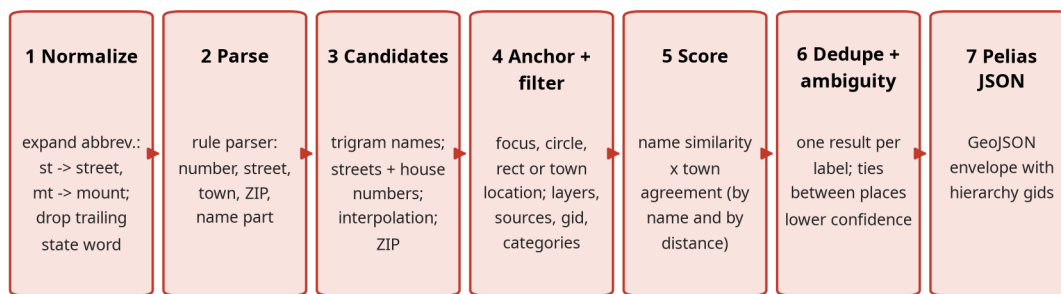


Figure 3. The pgeo forward-search pipeline. Each step is SQL; the tuning work in Section 2.7 changed steps 1, 3, 4, 5 and 6.

2.2 Data

Table 2. Data sources, raw input sizes, and what each engine holds from them. Pelias counts are Elastic-search documents; pgeo counts are rows in its feature table.

Source	Content	Raw input	Pelias documents	pgeo features
OpenAddresses	address points (Maine statewide E911 feed)	222 MB	780,260	677,229
OpenStreetMap	addresses, streets, venues (Geofabrik extract)	87 MB	768,992	128,471
Overture Maps	places (businesses, landmarks), 2026-08-19 release	16 MB	76,582	76,582
USGS GNIS	named features: lakes, summits, streams, populated places	4 MB	20,104	20,104
Who's On First	admin polygons: towns, counties, ZIPs, neighbourhoods	5,254 MB	3,280	3,289
Census ZCTA	ZIP code centroids	1 MB	426	426
Total			1,649,644	906,101

The two engines load the same inputs but store different numbers of records. The largest difference is addresses: Pelias holds **1,421,745** address documents and pgeo **702,701** (2.02 times as many in Pelias). OpenAddresses (the state's E911 address points) and OpenStreetMap both carry most Maine addresses; Pelias indexes both copies and removes duplicates when it answers a query, while pgeo removes them once at build time (same house number, street and town; OpenAddresses preferred). The data a user sees is the same; the index Pelias searches is roughly twice as large for addresses.

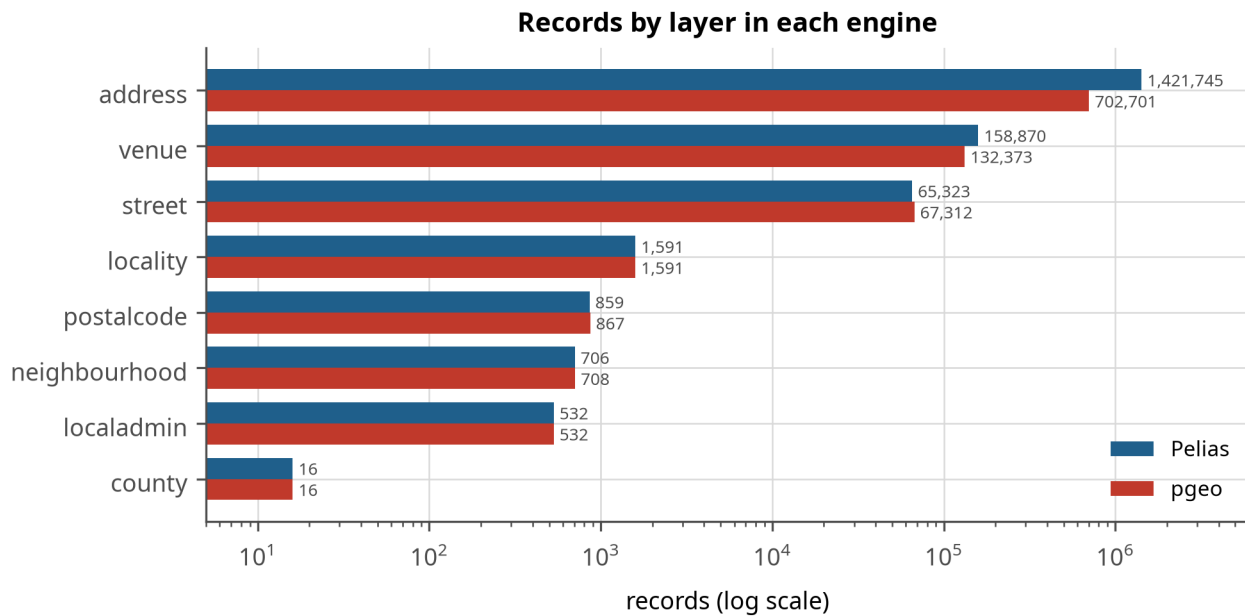


Figure 4. Records by layer in each engine (log scale). The address layer shows the build-time deduplication in pgeo; venues differ because pgeo also deduplicates OSM and Overture places that share a name and position.

Table 3. Records by layer.

Layer	Pelias	pgeo	pgeo / Pelias
address	1,421,745	702,701	0.49
venue	158,870	132,373	0.83
street	65,323	67,312	1.03
locality	1,591	1,591	1.00
localadmin	532	532	1.00
neighbourhood	706	708	1.00
postalcode	859	867	1.01
county	16	16	1.00
region	1	1	1.00
country	1	0	0.00

Data problems found and fixed during preparation (all in prep/): GNIS points outside Maine, a pipe-delimited ZCTA gazetteer, an Overture extract that leaked into Portsmouth, New Hampshire, and the Pelias interpolation builder needing OpenAddresses in its legacy CSV form.

2.3 Build pipelines

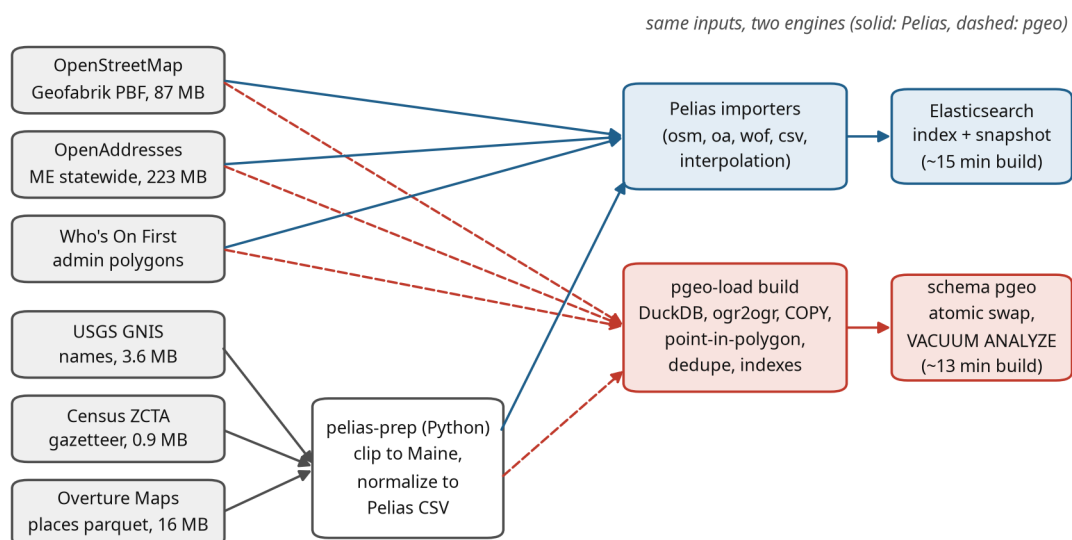


Figure 5. Data flow from the sources to each engine. Both engines read the same inputs; small sources pass through a preparation package that clips them to Maine.

The Pelias build (OSM, OpenAddresses, Who’s On First, CSV and interpolation importers) takes about 15 minutes on the workstation and produces a 435 MB index plus helper databases. The pgeo build (pgeo-load build) loads the same inputs with DuckDB, ogr2ogr and COPY, assigns each point its admin hierarchy by point-in-polygon, deduplicates, indexes, and swaps the new schema in atomically; it took 13 min in the latest run on one core.

Table 4. pgeo build steps and their duration (latest build).

Build step	Seconds
osm	43
stage	20
enrich_index	727

2.4 Accuracy testing

The accuracy test set was built from the source data, independently of both engines: the correct answer for each case is the source record’s position (for addresses, also the house number). A case counts as correct when the first result lies within the case’s radius (for example 50 m for an address, 8 km for a town); a “miss” case (a place that does not exist in Maine) counts as correct when the engine returns nothing, or nothing with confidence of at least 0.8.

Table 5. Composition of the accuracy test set by endpoint, category and query quality.

Endpoint	Category	Exact	Typo	Variant	Miss	Total
autocomplete	Towns	75				75
autocomplete	Venues	75				75
reverse	Reverse	200				200
search	Addresses	220	87	93		400
search	Lakes, summits	83	35	32		150
search	Misses				150	150
search	Towns	92	34	24		150

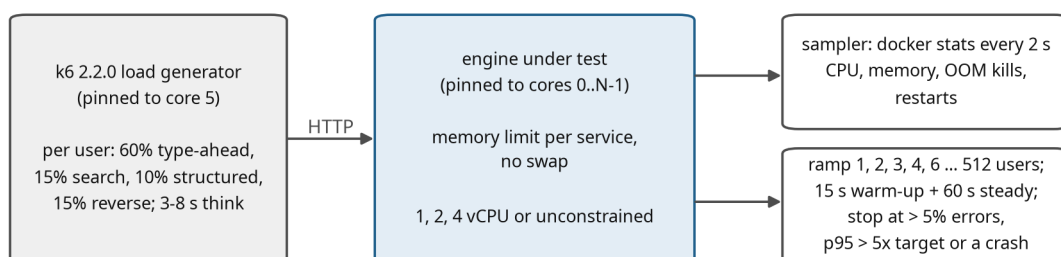
Endpoint	Category	Exact	Typo	Variant	Miss	Total
search	Venues	108	42			150
search	ZIP codes	60				60
structured	Addresses	150				150

Non-unique names (there are dozens of “Mud Pond”s in Maine) were qualified with the nearest town, because a bare “Mud Pond” has no single right answer. A separate **fuzz set** of 1,800 cases corrupts 300 exact queries at six levels:

Table 6. Fuzz levels. House numbers are never corrupted, so every query stays answerable.

Level	Corruption
F0	none (baseline)
F1	one character error (swap, drop, double, keyboard neighbour)
F2	two character errors in different words
F3	three errors, abbreviation flips (St <-> Street), commas dropped, lowercase
F4	F3 plus a component dropped (town, suffix or state) and word-order changes
F5	heavy: about 25% of letters corrupted, phonetic spellings (ph -> f, ck -> k)

2.5 Load and capacity testing



Host: AMD Ryzen 5 3600 (6 cores / 12 threads), 31 GB RAM, NVMe. The engine and the load generator never share a physical core.

Figure 6. Load-test harness. The load generator and the engine under test are pinned to different physical cores; each engine service runs with a memory limit and no swap.

Each simulated user behaves like a person using the demo page: 60% of sessions type an address or place letter by letter (autocomplete requests), 15% run a full search, 10% a structured search and 15% click the map (reverse), each followed by 3 to 8 seconds of think time. Text queries are 65% exact, 12% with typos, 13% off-name variants and 10% impossible places; 10% of map clicks fall offshore.

Table 7. Load-test query corpus (drawn at random per request).

Corpus pool	Items	Used for
addresses	3,000	type-ahead and search, exact form
places	800	towns, lakes, summits
venues	1,000	businesses and landmarks
zips	200	ZIP code searches
structured	1,000	structured search field sets
typos	1,200	one or two character errors

Corpus pool	Items	Used for
variants	1,200	off-name variants (abbreviations, missing town)
misses	600	places that do not exist in Maine
oa_points / random_points	1,000 / 1,000	reverse geocoding on land
miss_points	300	reverse geocoding offshore (misses)
foci	120	focus points (map centers) for type-ahead

Table 8. Example requests from the load session.

Endpoint	Example request (as sent by the load generator)
Autocomplete	/v1/autocomplete?text=389 cong&size=8&focus.point.lat=43.66&focus.point.lon=-70.26
Search	/v1/search?text=389 Congress St, Portland, ME&size=8&focus.point.lat=...
Search (typo)	/v1/search?text=Moosehed Lake&size=8
Structured	/v1/search/structured?address=389 Congress St&locality=Portland®ion=ME&postalcode=04101&size=8
Reverse	/v1/reverse?point.lat=43.6568&point.lon=-70.2626&size=6

Table 9. Latency targets (95th percentile) used as the pass criterion.

Endpoint	p95 target	Why
Autocomplete	250 ms	type-ahead must keep up with typing
Search	750 ms	a full search after pressing Enter
Structured search	750 ms	form-based search
Reverse	400 ms	map click; feels immediate
All	errors < 1%	no restarts or out-of-memory kills during a run

For every configuration the harness recreates the services with their CPU and memory limits, warms them up, runs a 3-user validation (3 minutes), then ramps through 1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 48, 64, 96, 128, 192, 256, 384 and 512 users (15 s warm-up plus 60 s measured at each step). A step **passes** when every endpoint's p95 is within target and errors stay below 1%; the run stops when errors exceed 5%, any p95 exceeds five times its target, or a service dies. Caches warm up by design and warm-up traffic is excluded, so the numbers describe a service in steady use, not a cold start.

Table 10. Tested configurations. pgeo configurations were run with each front end (rest = pure SQL through PostgREST, api = FastAPI); budgets include 0.8 GB for the operating system.

Engine	Config	vCPU	Memory budget	Notes
Pelias	M0	all	unlimited	reference, 1 API worker
Pelias	C1 / C1s	1	8.3 / 9.1 GB	floor and standard memory profiles
Pelias	C2 / C3	2	8.4 / 9.5 GB	floor / standard
Pelias	C4 / C4a	4	10.8 / 10.4 GB	4 API workers / 1 API worker
pgeo	Pmin	1	1.6 GB	database 0.6 GB: smaller than the data
pgeo	P1	1	2.0 GB	database 1.0 GB
pgeo	P2	2	2.5-2.7 GB	database 1.5 GB
pgeo	P4	4	3.0-3.3 GB	database 2.0 GB
pgeo	PM	all	unlimited	reference

2.6 Compatibility testing

A contract test (`tests/compat/compat_test.py`) sends 27 documented Pelias requests to Pelias and to both pgeo front ends and checks each response for status code, envelope shape, property names, filter behaviour (results inside the requested circle, rectangle or area; no results for another country) and Pelias-shaped errors. Pelias is the reference: a check that Pelias itself fails is reported but not counted against pgeo.

2.7 Tuning procedure

pgeo was tuned in rounds. Each round started from the failing test cases (what came back, what was expected, which query step lost the right answer), changed one thing, and re-ran the full accuracy set, the fuzz set and a per-category comparison that listed every flipped case. A change was kept only if overall accuracy did not drop and no category lost more than noise. The final state is protected by:

- **tuning profiles** (`pgeo-tune`): measured PostgreSQL and front-end settings per machine size, applied and verified against the running server;
- an **accuracy gate** (`tests/accuracy/gate.py`) that fails a build if accuracy, any category, calibration or any fuzz level regresses beyond small tolerances;
- a **rebuild pipeline** (`scripts/pgeo_rebuild.sh`): build, profile, known-answer checks, gate.

3. Results

3.1 Accuracy

Table 11. Accuracy on 1,560 cases. Correct = right answer at rank 1 (misses: nothing, or low confidence); Hit@5 = right answer in the top five; confidence columns are the mean confidence of the first result when right and when wrong.

Engine	Correct	Hit@1	Hit@5	Exact	Typo	Variant	Miss	Conf. right / wrong	p50 ms
Pelias	75.5%	77.0%	83.5%	88%	30%	85%	41%	0.98 / 0.87	13
pgeo pure SQL	95.8%	94.4%	98.2%	98%	87%	95%	95%	0.92 / 0.69	39
pgeo FastAPI (rule parser)	95.8%	94.4%	98.2%	98%	87%	95%	95%	0.92 / 0.69	46
pgeo FastAPI (libpostal service)	94.0%	92.4%	96.9%	95%	87%	93%	94%	0.92 / 0.60	45

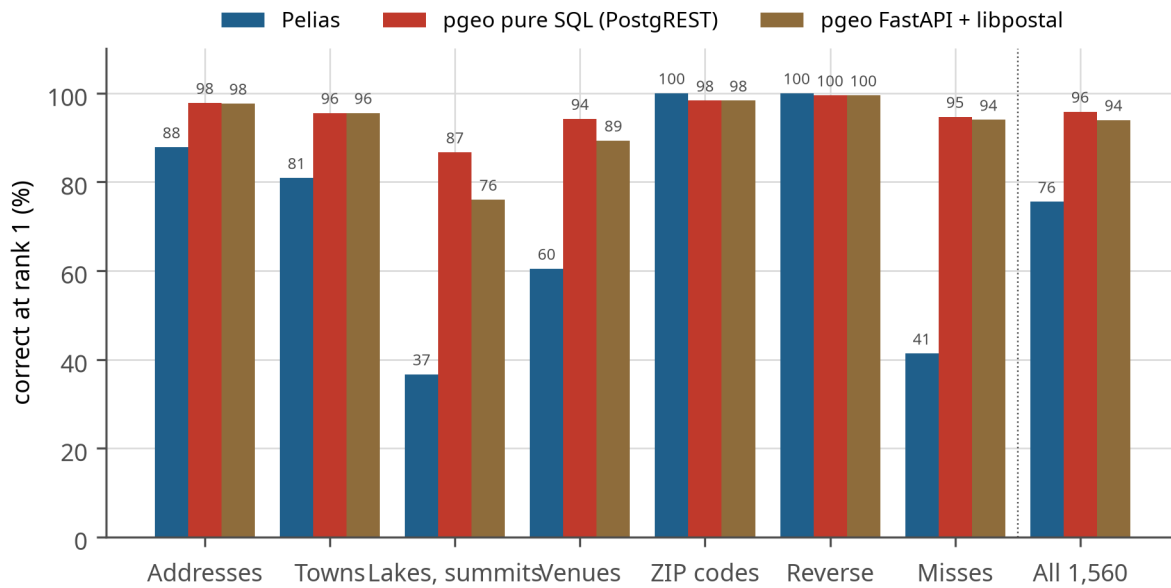


Figure 7. Correct answers by category. pgeo leads in every category except ZIP codes and reverse geocoding, where both engines are near perfect.

Table 12. Accuracy by category (all query qualities).

Category	Cases	Pelias	pgeo pure SQL	pgeo FastAPI (rule parser)	pgeo FastAPI (libpostal service)
Addresses	550	88%	98%	98%	98%
Towns	225	81%	96%	96%	96%
Lakes, summits	150	37%	87%	87%	76%
Venues	225	60%	94%	94%	89%
ZIP codes	60	100%	98%	98%	98%
Reverse	200	100%	100%	100%	100%
Misses	150	41%	95%	95%	94%

The categories where pgeo gains most are the ones where the query and the data use different words for the same place. Pelias ranks text with Elasticsearch’s term matching over analysed tokens; a misspelled token matches nothing, and its admin lookup (placeholder) needs exact town names. pgeo compares whole strings by trigram similarity and treats the queried town as a **location** rather than a name to match exactly: “Calvary Bible Church, Stratton” is found although the church lies in Eustis, because it is near Stratton. That single change raised venue accuracy from 81% to 94%.

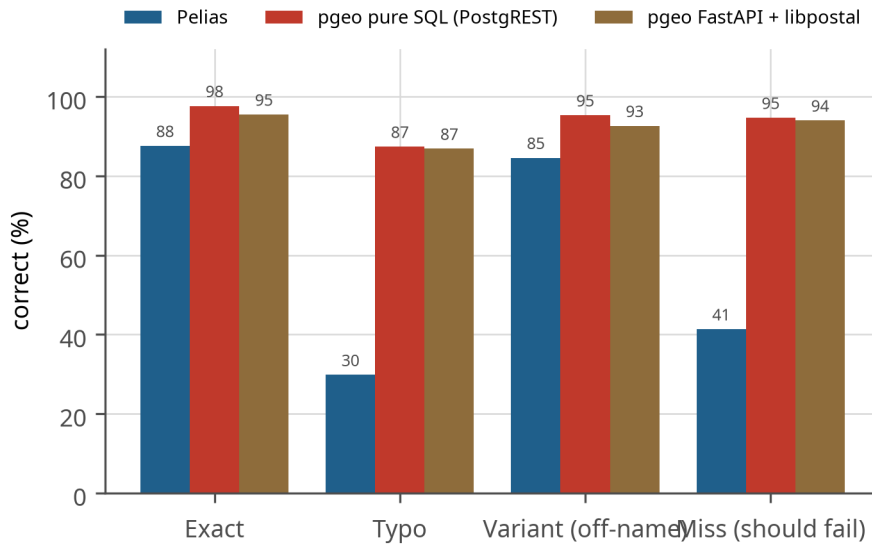


Figure 8. Accuracy by query quality. Typos separate the engines most; “miss” means the engine correctly returned nothing or only low-confidence results for a place that does not exist.

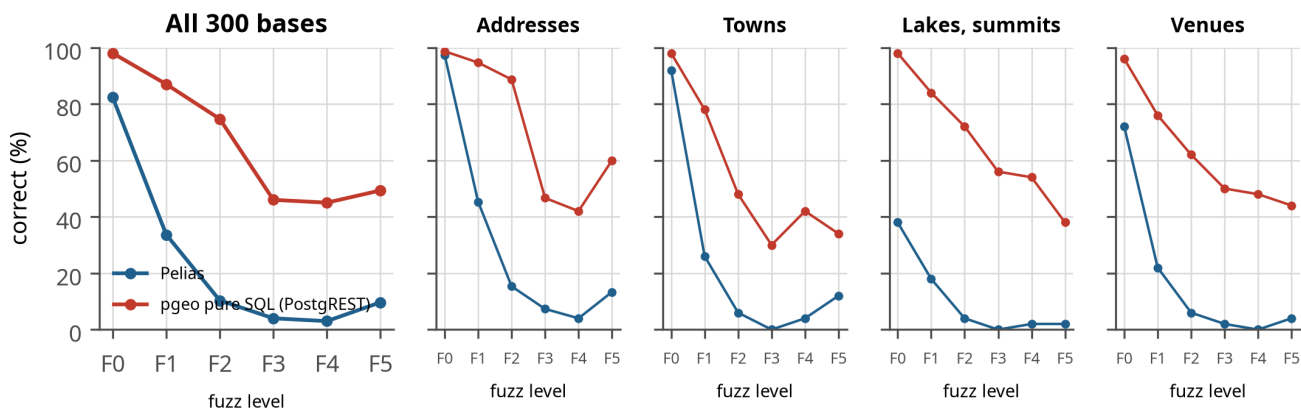


Figure 9. Accuracy as queries are progressively corrupted (fuzz levels F0-F5), overall and by category. Pelias falls to near zero from two character errors; pgeo degrades gradually.

Table 13. Fuzz results (correct, %).

Engine	F0	F1	F2	F3	F4	F5
Pelias	82%	34%	10%	4%	3%	10%
pgeo pure SQL	98%	87%	75%	46%	45%	49%

Calibration. A confidence score is useful only if wrong answers get lower scores than right ones. pgeo’s first-result confidence averages 0.92 when right and 0.69 when wrong; Pelias scores 0.98 and 0.87, so a client can use pgeo’s confidence to decide when to ask the user.

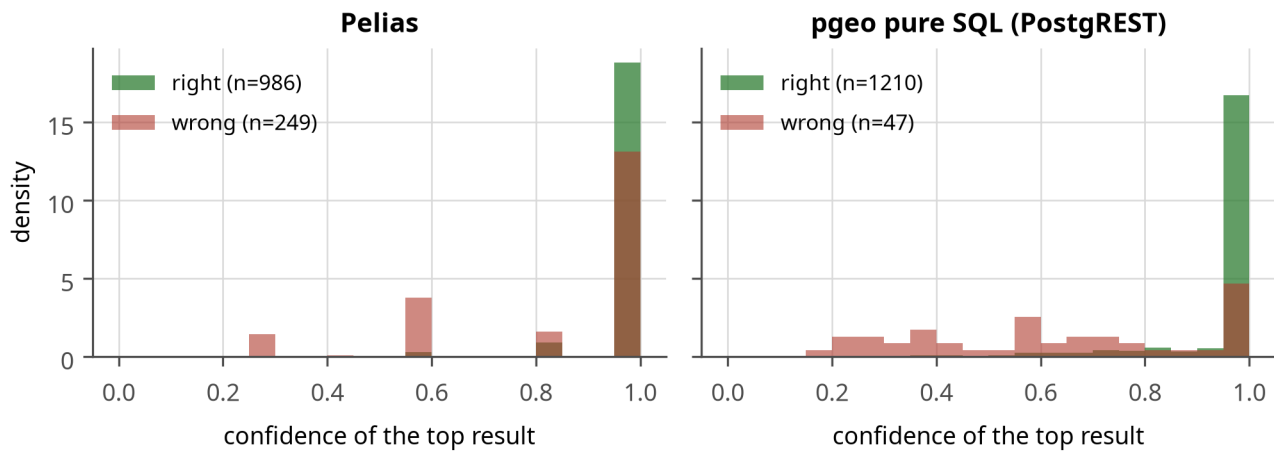


Figure 10. Distribution of first-result confidence for right and wrong answers (misses excluded). pgeo separates the two groups more clearly.

Remaining failures. 65 of 1,560 cases still fail on pgeo (Table 14). Eight “misses” are test-set errors (Death Valley in Minot and “The Eiffel Tower of Paris, Maine” exist); about ten are genuinely ambiguous (chain branches, two towns with the same name); short autocomplete prefixes without a focus point (“Nor”, “Tim Ho”) cannot be resolved by any engine; the hardest remaining class is typos in both the place name and the town.

Table 14. Remaining pgeo failures by category.

Category (query quality)	Failures
Lakes, summits (typo)	14
Venues (exact)	11
Misses (miss)	8
Addresses (typo)	5
Towns (exact)	5
Addresses (variant)	4
Towns (typo)	4
Lakes, summits (exact)	4
Addresses (exact)	3
Lakes, summits (variant)	2
Venues (typo)	2
Reverse (exact)	1
Towns (variant)	1
ZIP codes (exact)	1

3.2 Tuning trajectory

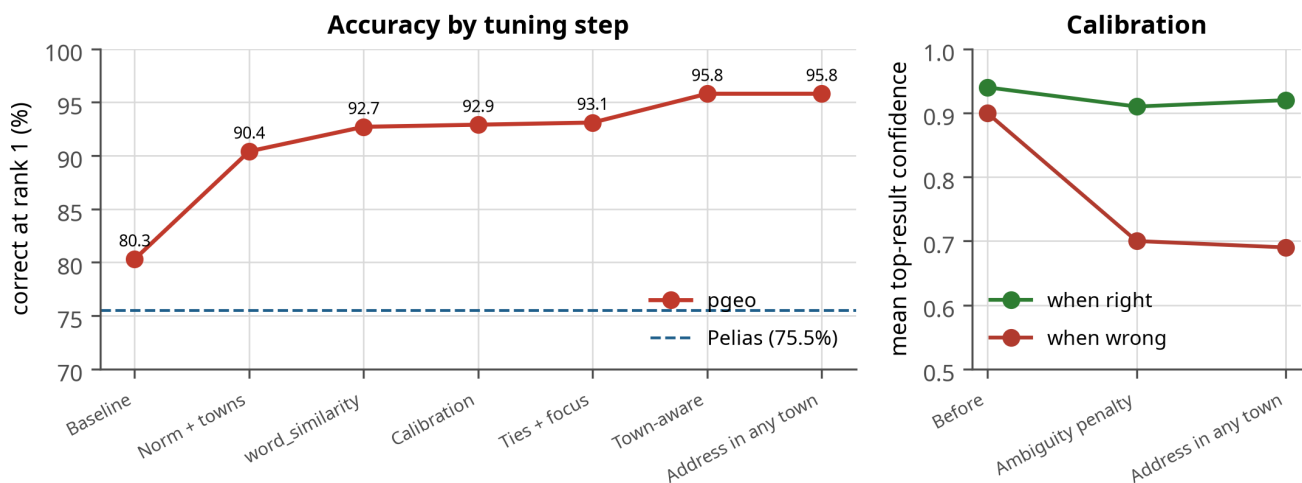


Figure 11. Left: pgeo accuracy after each tuning step, with Pelias for reference. Right: first-result confidence when right and when wrong; the ambiguity penalty widened the gap.

Table 15. Tuning changes, the problem each solved, and the measured effect.

Step	Change	Problem solved	Effect
Normalization	Abbreviations expanded (rd -> road, st -> street or saint)	typos in spelled-out words failed (similarity 0.25 vs 0.81)	part of 80.3% -> 90.4%
Towns	towns matched on WOF and postal city; ZIP only a fallback	structured search 0% at rank 1 (ZIP outranked the address)	structured 0% -> 100%
Dedupe	OA/OSM addresses merged; results one per label	duplicated towns and addresses	clean lists
word_similarity	direction (query, name)	"Mountain" matched every query containing the word	lakes/summits 59% -> 77%
Calibration	confidence divided among tied distinct places	wrong answers were mostly ties (70 of 85)	confidence wrong 0.90 -> 0.70
Ties + focus	candidates ordered by distance to the focus before the row limits	hundreds of "Main Street"s cut to 40 before the focus applied	"main st" near Bangor returns Bangor
Town-aware	the queried town is a location (distance-based agreement)	village or nearest town given instead of the containing one	venues 81% -> 94%; 93.1% -> 95.8%
Address in any town	house number looked up on the best street names statewide	25 of 150 "Church Street" towns kept arbitrarily	confidence wrong 0.73 -> 0.69
Reverse index	index on the admin join	every reverse request scanned all features	see Section 3.4

3.3 Latency at the 3-user target

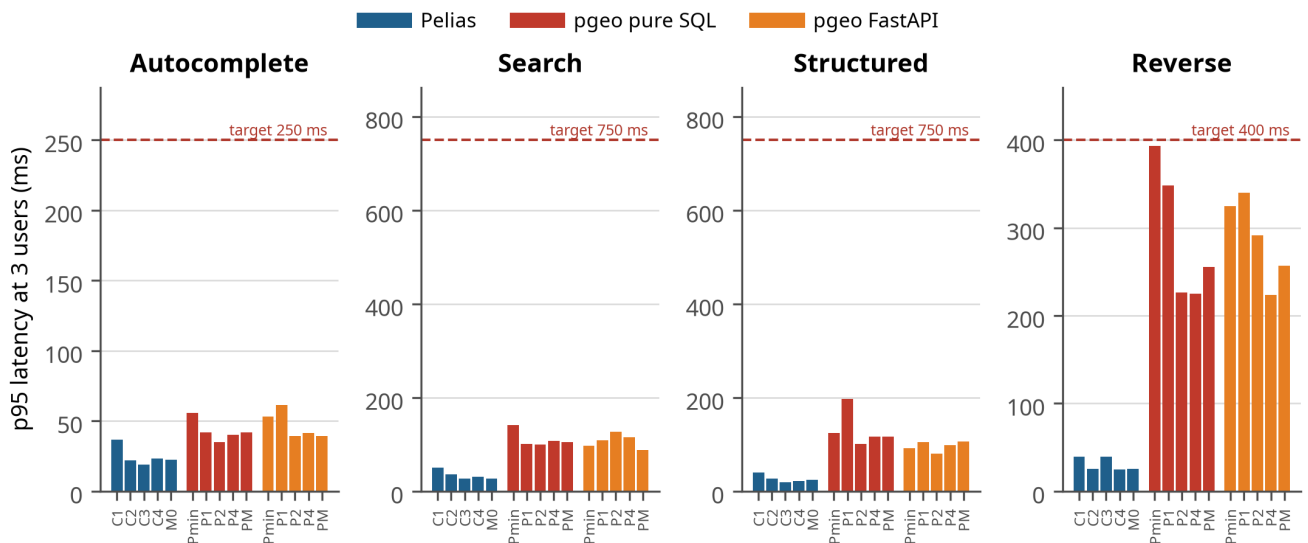


Figure 12. p95 latency per endpoint at 3 users for each configuration, against its target (dashed). Every configuration of both engines meets every target.

Table 16. Median and p95 latency at 3 users (ms).

Config	Autocomplete p50 / p95 ms	Search p50 / p95 ms	Structured p50 / p95 ms	Reverse p50 / p95 ms
C1	15 / 37	26 / 50	28 / 40	24 / 40
C2	9 / 22	20 / 36	19 / 27	19 / 26
C3	8 / 19	14 / 27	13 / 20	10 / 39
C4	8 / 23	21 / 30	17 / 22	22 / 25
M0	8 / 23	20 / 27	20 / 25	20 / 25
rest-Pmin	22 / 56	59 / 142	90 / 124	311 / 393
rest-P1	16 / 42	50 / 101	103 / 197	321 / 348
rest-P2	15 / 35	40 / 100	89 / 100	213 / 226
rest-P4	15 / 40	37 / 107	93 / 116	190 / 225
rest-PM	14 / 42	38 / 106	91 / 116	235 / 255
api-Pmin	20 / 53	30 / 98	59 / 91	293 / 325
api-P1	19 / 61	78 / 109	83 / 104	277 / 340
api-P2	16 / 39	46 / 127	44 / 81	282 / 291
api-P4	14 / 41	49 / 116	89 / 99	199 / 224
api-PM	15 / 39	48 / 89	93 / 106	232 / 257
api-svc-P2	16 / 46	48 / 109	79 / 116	212 / 225
api-svc-P4	17 / 45	49 / 108	99 / 106	209 / 232

3.4 Capacity under load

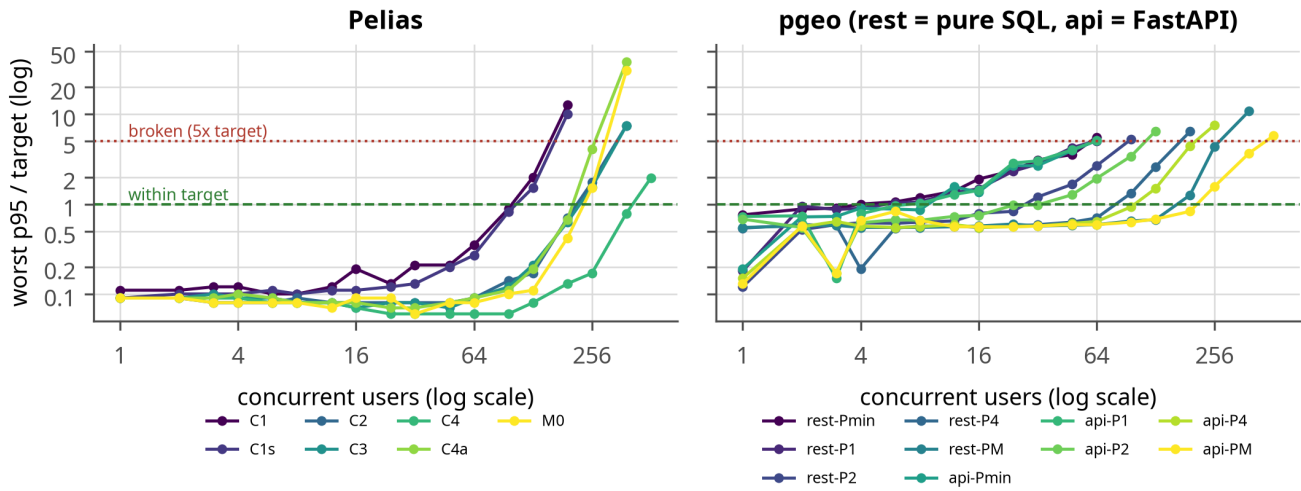


Figure 13. Worst endpoint p95 relative to its target as users are added. Above 1 the configuration misses a target; above 5 the run stops. At 1-4 users some endpoints receive only a few requests per window, so single points are noisy.

Table 17. Pelias capacity by configuration.

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
C1	1	8.3 GB	heap 768m, 1 worker(s)	96	192	autocomplete, search, reverse at 128	99	5.9 GB
C1s	1	9.1 GB	heap 768m, 1 worker(s)	96	192	autocomplete, reverse at 128	102	5.8 GB
C2	2	8.4 GB	heap 768m, 2 worker(s)	192	384	autocomplete, reverse at 256	209	6.1 GB
C3	2	9.5 GB	heap 1g, 2 worker(s)	192	384	autocomplete, reverse at 256	213	6.3 GB
C4	4	10.8 GB	heap 2g, 4 worker(s)	384	not reached	autocomplete, search, structured, reverse at 512	403	7.5 GB
C4a	4	10.4 GB	heap 2g, 1 worker(s)	192	384	autocomplete, search, structured, reverse at 256	194	7.1 GB
M0	all	-	heap 4g, 1 worker(s)	192	384	autocomplete, reverse at 256	215	9.3 GB

Table 18. pgeo capacity by configuration (after the reverse-geocoding index fix).

Config	vCPU	Budget	Settings	Users within SLO	Broken at	First over target	req/s at limit	Memory at 3 users
rest-Pmin	1	1.6 GB	sb 128MB, 4 conn.	4	64	reverse at 6	4	0.3 GB
rest-P1	1	2.0 GB	sb 256MB, 4 conn.	4	64	reverse at 6	4	0.4 GB
rest-P2	2	2.5 GB	sb 384MB, 6 conn.	24	96	reverse at 32	26	0.5 GB
rest-P4	4	3.0 GB	sb 512MB, 10 conn.	64	192	reverse at 96	73	0.5 GB
rest-PM	all	-	sb 1GB, 16 conn.	128	384	reverse at 192	145	0.6 GB
api-Pmin	1	1.6 GB	sb 128MB, 4 conn.	8	64	reverse at 12	10	0.3 GB
api-P1	1	2.0 GB	sb 256MB, 4 conn.	6	64	reverse at 8	8	0.4 GB
api-P2	2	2.7 GB	sb 384MB, 6 conn.	32	128	reverse at 48	37	0.6 GB
api-P4	4	3.3 GB	sb 512MB, 10 conn.	96	256	reverse at 128	108	0.8 GB
api-PM	all	-	sb 1GB, 16 conn.	192	512	autocomplete, reverse at 256	218	0.8 GB
api-svc-P2	2	4.7 GB	sb 384MB, 6 conn.	24	128	reverse at 32	29	2.5 GB
api-svc-P4	4	5.3 GB	sb 512MB, 10 conn.	32	32	-	29	2.7 GB

Table 19. Effect of the reverse-geocoding index on pgeo capacity and reverse latency.

Config	Users within SLO,		Reverse p95 at 3 users	
	before	after	(ms), before	after
rest-Pmin	4	4	393	393
rest-P1	4	4	348	348
rest-P2	24	24	226	226
rest-P4	64	64	225	225
rest-PM	128	128	255	255
api-Pmin	8	8	325	325
api-P1	6	6	340	340
api-P2	32	32	291	291
api-P4	96	96	224	224
api-PM	192	192	257	257

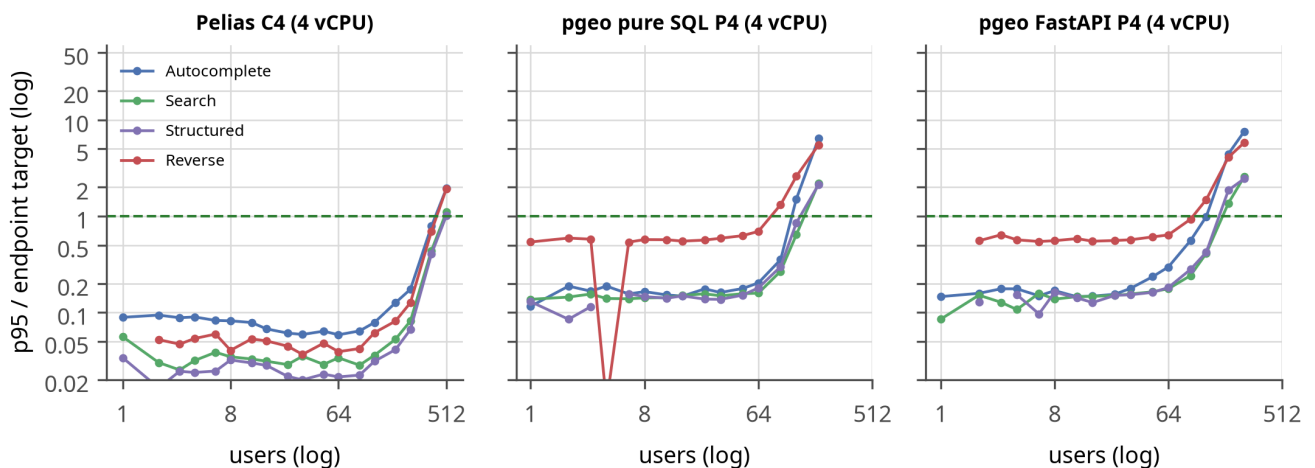


Figure 14. p95 per endpoint relative to its target on 4 vCPU. The first line to cross 1 is the endpoint that limits capacity.

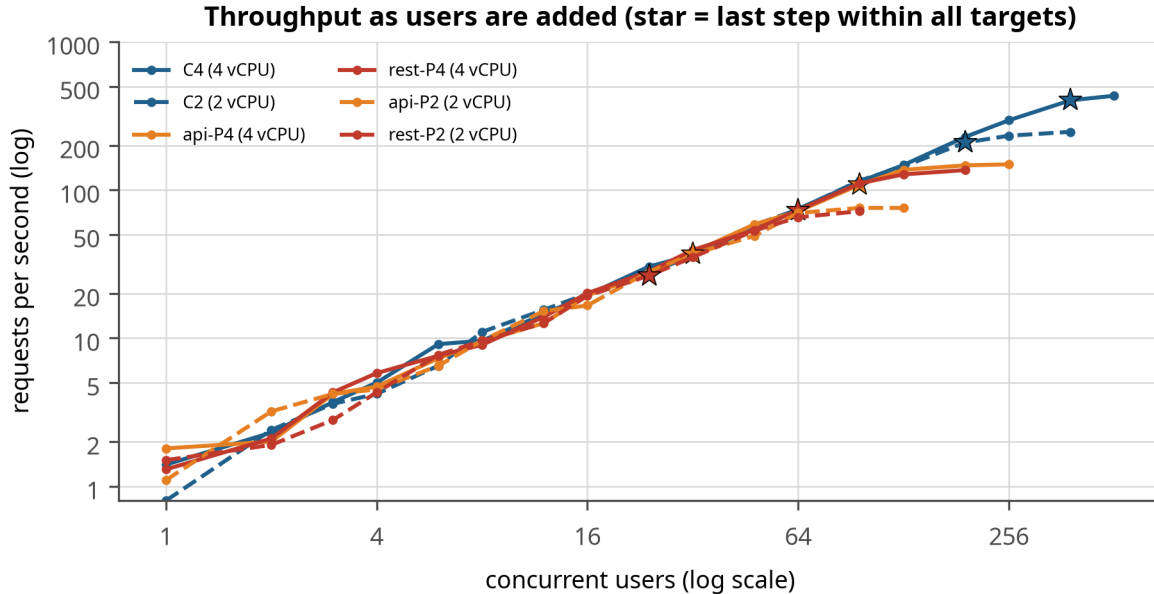


Figure 15. Throughput as users are added; the star marks the last step within all targets.

Pelias spreads work across services in different languages: Elasticsearch does the text and geo matching in compiled code, and the Node.js API adds parsing and ranking. With four API workers on 4 vCPU (C4) it reaches 384 users; with one worker on the same CPUs (C4a) only 192, because a single Node.js worker becomes the bottleneck.

3.5 Resources

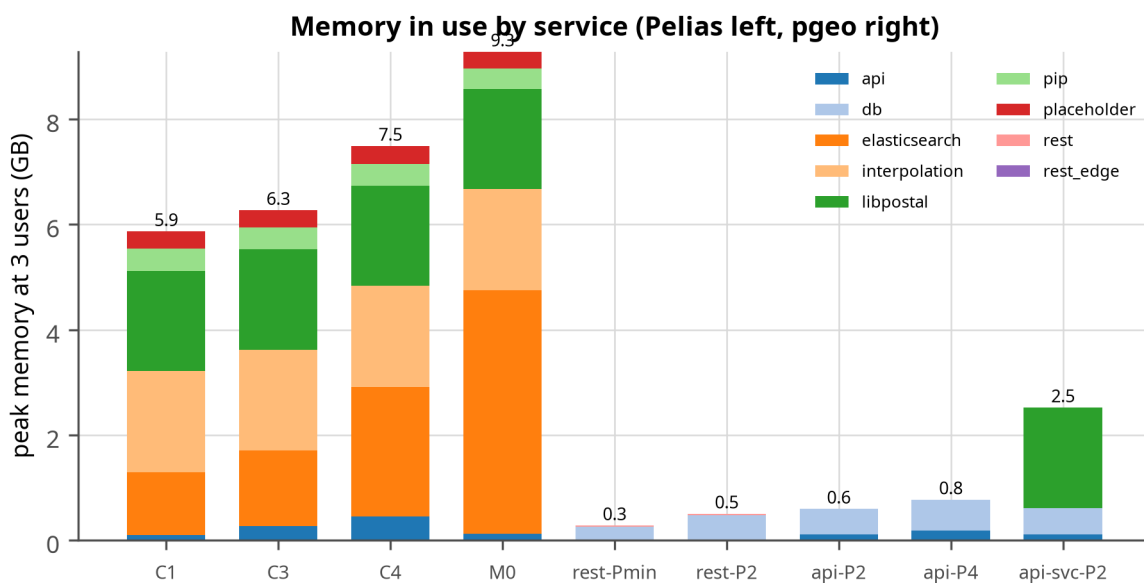


Figure 16. Peak memory in use by service at 3 users. Pelias carries the libpostal model (~2 GB), the interpolation database and Elasticsearch's heap regardless of load; pgeo's memory is mostly the PostgreSQL page cache for a 674 MB database.

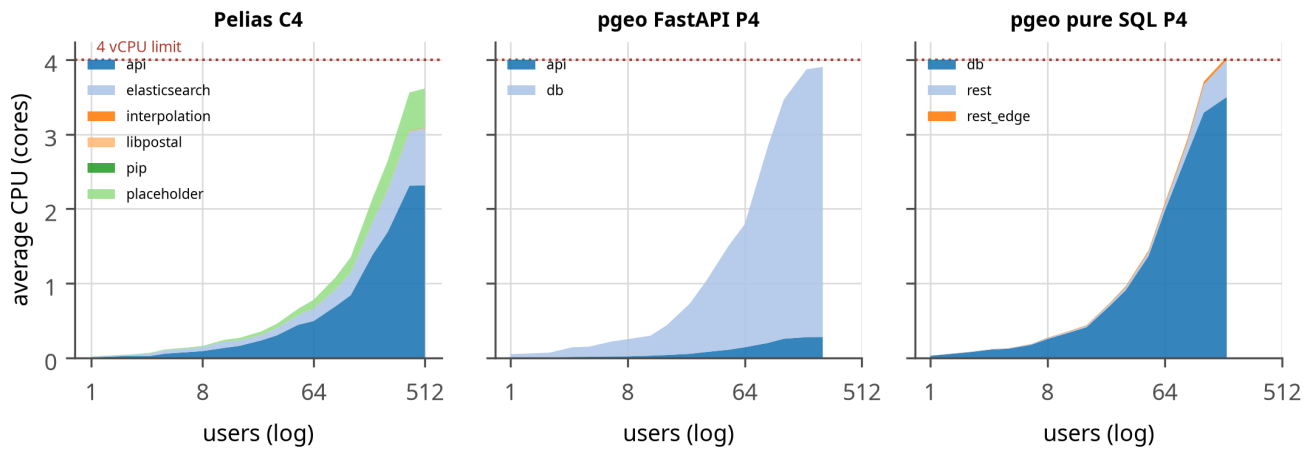


Figure 17. Average CPU by service as users are added on 4 vCPU.

3.6 Data volume

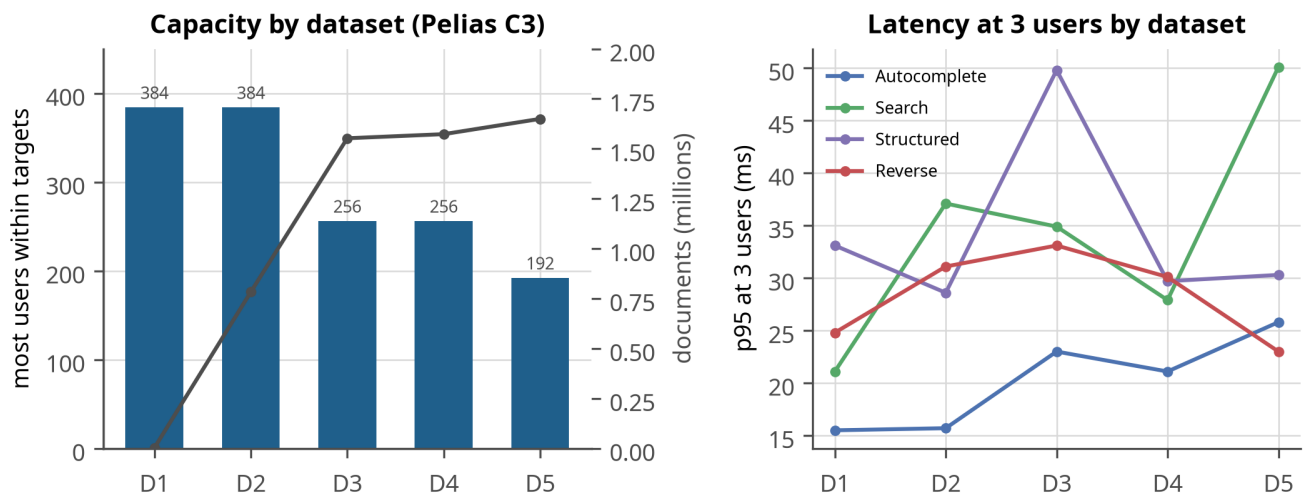


Figure 18. Pelias capacity (bars) and document count (line) for cumulative datasets D1-D5 at fixed resources (2 vCPU, 9.5 GB), and p95 latency at 3 users.

Table 20. Pelias by dataset: D1 Who's On First; D2 + OpenAddresses; D3 + OpenStreetMap; D4 + GNIS and ZCTA; D5 + Overture places (full).

Dataset	Documents	Index	Users within SLO	Broken at	Autocomplete p95	Search p95	Structured p95	Reverse p95
D1	3,280	3 MB	384	not reached	16	21	33	25
D2	783,540	196 MB	384	not reached	16	37	29	31
D3	1,552,532	371 MB	256	512	23	35	50	33
D4	1,573,062	378 MB	256	512	21	28	30	30
D5	1,649,644	435 MB	192	384	26	50	30	23

Adding 780,000 OpenAddresses documents (D2) changed nothing measurable: exact address terms are cheap to match. Adding OpenStreetMap (D3) and Overture places (D5) each cost one ramp step, because

venue and place names are the documents that broad text queries match: more of them means more candidates to score per query.

3.7 Query types

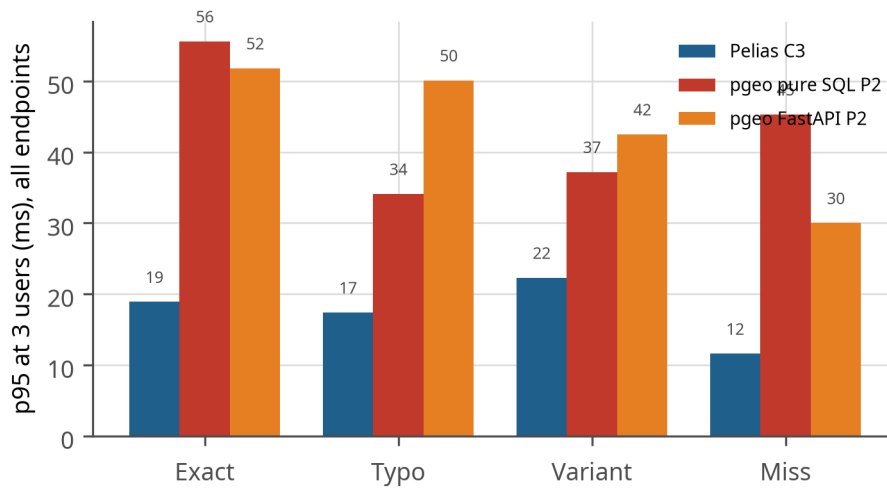


Figure 19. p95 latency at 3 users by query quality (all endpoints together).

3.8 Compatibility

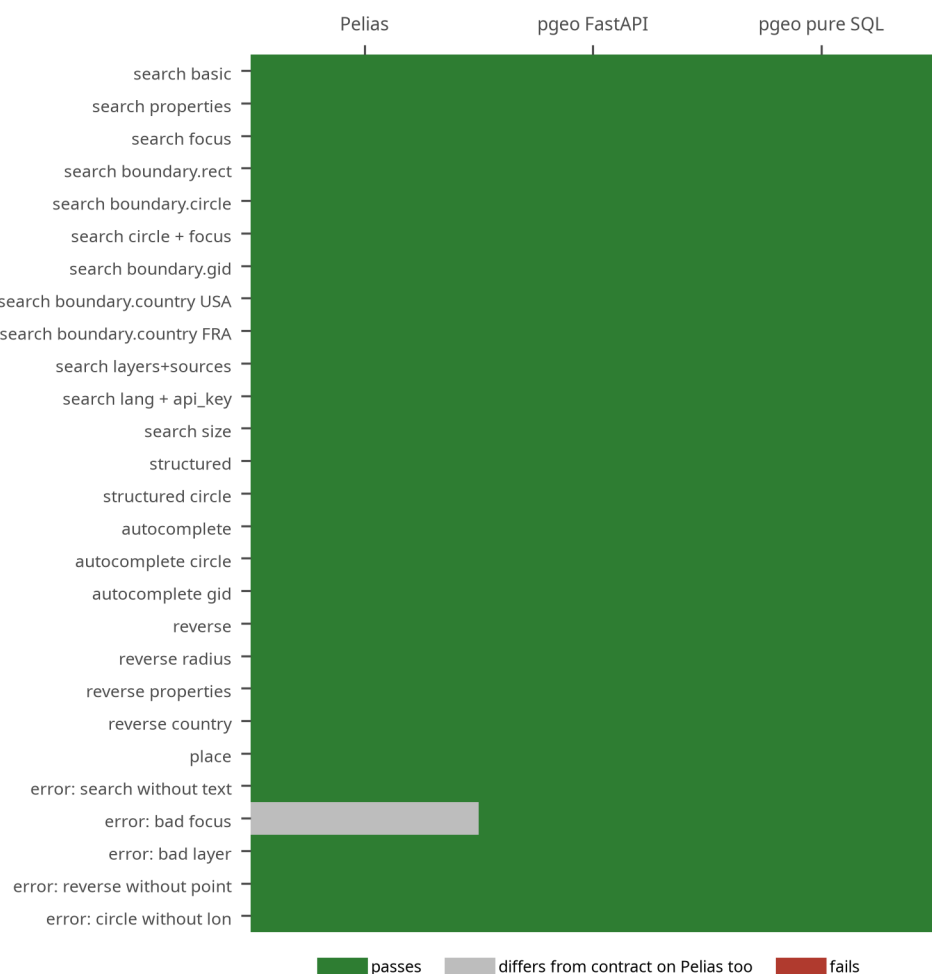


Figure 20. Compatibility contract results: each row is a documented Pelias request, each column an engine.

Table 21. Compatibility contract cases.

Case	Request	Pelias	pgeo FastAPI	pgeo pure SQL
search basic	/v1/search?text=389 Congress St, Portland, ME	pass	pass	pass
search properties	/v1/search?text=389 Congress St, Portland, ME	pass	pass	pass
search focus	/v1/search?text=main st&focus.point.lat=44.8016&focus.point.lon=- 68.7712	pass	pass	pass
search boundary.rect	/v1/search?text=main st&boundary.rect.min_lat=44.7&boundary.rect.max_lat=44.9&boundary.re	pass	pass	pass
search boundary.circle	/v1/search?text=main st&boundary.circle.lat=44.8016&boundary.circle.lon=- 68.7712&boundary	pass	pass	pass
search circle + focus	/v1/search?text=main st&boundary.circle.lat=44.8016&boundary.circle.lon=- 68.7712&boundary	pass	pass	pass
search boundary.gid	/v1/search?text=congress st&boundary.gid=whosonfirst:locality:85948877	pass	pass	pass

Case	Request	Pelias	pgeo FastAPI	pgeo pure SQL
search	/v1/search?text=bangor&boundary.country=USA	pass	pass	pass
boundary.country USA				
search	/v1/search?text=bangor&boundary.country=FRA	pass	pass	pass
boundary.country FRA				
search layers+sources	/v1/search?text=portland&layers=locality&sources=wof	pass	pass	pass
search lang + api_key	/v1/search?text=bangor&lang=en&api_key=abc123	pass	pass	pass
search size	/v1/search?text=main st&size=3	pass	pass	pass
structured	/v1/search/structured?address=389 Congress St&locality=Portland®ion=ME	pass	pass	pass
structured circle	/v1/search/structured?address=Main St&boundary.circle.lat=44.8016&boundary.circle.lon=-68	pass	pass	pass
autocomplete	/v1/autocomplete?text=389 congress	pass	pass	pass
autocomplete circle	/v1/autocomplete?text=main&boundary.circle.lat=44.8016&boundary.circle.lon=-68.7712&bound	pass	pass	pass
autocomplete gid	/v1/autocomplete?text=congress&boundary.gid=whosonfirst:locality:85948877	pass	pass	pass
reverse	/v1/reverse?point.lat=43.6568&point.lon=-70.2626	pass	pass	pass
reverse radius	/v1/reverse?point.lat=43.6568&point.lon=-70.2626&boundary.circle.radius=1	pass	pass	pass
reverse properties	/v1/reverse?point.lat=43.6568&point.lon=-70.2626&layers=address	pass	pass	pass
reverse country	/v1/reverse?point.lat=43.6568&point.lon=-70.2626&boundary.country=US	pass	pass	pass
place	/v1/place?ids=whosonfirst:locality:85948877	pass	pass	pass
error: search without text	/v1/search?	pass	pass	pass
error: bad focus	/v1/search?text=x&focus.point.lat=0&focus.point.lon=0	pass	pass	pass
error: bad layer	/v1/search?text=x&layers=planet	pass	pass	pass
error: reverse without point	/v1/reverse?	pass	pass	pass
error: circle without lon	/v1/search?text=x&boundary.circle.lat=44.8	pass	pass	pass

Before this work pgeo failed 28 contract checks; the fixes added `boundary.circle`, `boundary.gid`, `boundary.country` and categories filtering, the Pelias hierarchy identifiers (`locality_gid`, `county_gid`, ...), Pelias-shaped error responses from the pure-SQL path, and an edge rule that renames `boundary.circle.lat/lon` (which PostgREST would otherwise read as the focus point's lat/lon). Remaining differences (docs/PELIAS_COMPATIBILITY.md): OpenAddresses records have different identifiers in the two engines, categories uses the sources' own category names, and the pure-SQL path rejects parameters it does not know where Pelias ignores them.

3.9 Features

Table 22. Features of each platform as deployed here.

Capability	Pelias	pgeo
Endpoints	search, structured, autocomplete, reverse, place (+ nearby, beta)	the same five, plus /v1/address
Filters	layers, sources, focus, rectangle, circle, country, gid, categories	the same
Parsing	libpostal (statistical, ~2 GB model)	rule parser in SQL (best measured); libpostal optional

Capability	Pelias	pgeo
Typo tolerance	limited (term fuzziness)	trigram similarity on whole names and streets
Town handling	exact admin names (placeholder)	town as a location: villages and nearest towns accepted
Interpolation	separate service (TIGER + OA)	SQL, between neighbouring house numbers
Confidence	per result	per result, lowered for ties between distinct places
Structured US addresses	no	USPS Publication 28 components, delivery and last lines, FIPS codes
Nearest street address for a venue	no	yes (/v1/address)
HTTP layer	Node.js API	PostgREST (no code) or FastAPI
Data store	Elasticsearch index	PostgreSQL tables
Components	6 containers	2-3 containers
Build	importers, ~15 min	one command, ~13 min
Operations tooling	Pelias CLI	tuning profiles, accuracy gate, rebuild script
Security posture here	read-only query services, pinned images, strict CSP	read-only database role, allowlisted tuning, no parameter logging

The demo page exposes these features: autocomplete with map-centre bias, structured search, reverse geocoding by map click, a CSV batch tool, and, where both engines are deployed, an engine switch, an Address tab (type-ahead with a best-match suggestion, the USPS block with a copy button, and the nearest street address drawn on the map for venues) and a Compare tab that runs one query on both engines side by side.

3.10 Production validation on VM 120

4. Discussion

Why pgeo is more accurate. The difference is in how text meets data. Elasticsearch matches analysed tokens; a token with a typo or an unexpected abbreviation contributes nothing, and the admin lookup requires exact names. pgeo compares normalized whole strings by trigram similarity, expands abbreviations before comparing (so “moountain rd” meets “mountain road”), and scores the queried town by distance as well as by name. It also knows the data is one state, which lets it use tie-breaks (distance to the focus point or the named town) that a planet-scale engine cannot assume.

Why Pelias is faster. Elasticsearch executes text matching as compiled index lookups and caches aggressively, and Pelias parallelizes across services. pgeo’s PL/pgSQL runs several candidate queries and scoring steps per request in an interpreted language inside one backend.

Threats to validity.

- The accuracy set comes from the same sources both engines load; it measures finding what is in the data, not whether the data is correct. Eight miss cases are test-set errors.
- Load tests ran on one workstation with CPU pinning and memory limits standing in for VM sizes; absolute numbers on other hardware differ (Section 3.10 validates on the production VM).
- At 1-4 users the per-endpoint p95 rests on few requests; the ramp’s decisions at higher user counts rest on thousands.
- One pgeo run was interrupted by a host event (the Docker daemon was stopped at 04:32 on 2026-09-19, unrelated to the test); it is marked where it appears.
- The first Pelias data-volume run failed because Elasticsearch was not ready after the previous run’s restart; it was repeated with a readiness wait.

5. Conclusions

Table 23. Which tool for which scenario.

Scenario	Recommended	Why
A few users on a small VM or VPS (this project's target)	pgeo, pure SQL	meets all targets in 1.6-3.3 GB; most accurate; 2-3 containers
Messy input: typos, variants, venues, impossible places	pgeo	87% vs 30% on typos; misses handled 95% vs 41%
Structured US addresses for another system (LANCER)	pgeo	/v1/address; nearest street address for venues
Hundreds of concurrent users per vCPU	Pelias	4-24x the throughput per CPU
Multi-state or national coverage	Pelias today	built for planet scale; pgeo's per-query cost grows with candidates
Fewest components, data in PostgreSQL (G6)	pgeo, pure SQL	database plus a stateless gateway
Existing Pelias clients	either	pgeo passes the compatibility contract

6. Next steps

- **Reverse and autocomplete speed:** profile the remaining per-request cost in PL/pgSQL (prepared statements, fewer candidate queries, a GiST trigram index for ordered similarity).
- **Scale out for more users:** read replicas behind PostgREST, or caching of frequent queries at the edge; measure against the Pelias curve.
- **New Hampshire:** build both engines for two states; measure the data-volume effect on pgeo (subset builds) as was done for Pelias.
- **Category taxonomy:** map source categories to Pelias's taxonomy so categories=food works.
- **Test set:** remove the eight false misses; add real user queries once the service is in use.
- **libpostal model update** (Senzing libpostal-data): re-test the libpostal arm for accuracy and memory.
- **Authorization** (Phase 11): single sign-on plus hashed API keys at the edge before LANCER uses the service; strip query strings from access logs.
- **Continuous checks:** run the accuracy gate and the compatibility contract on every build.

Appendix A. Reproducing this report

```
scripts/build_report.sh # rebuild from saved results
scripts/build_report.sh --refresh # refresh live snapshot and contract test first
scripts/build_report.sh --refresh-accuracy # also re-run the accuracy and fuzz sets
```

Inputs: report/inputs.toml. Figures are published as PNG (used in this report) and SVG (editable) in docs/report_figures/. Load tests: tests/load/run_matrix.py (Pelias) and tests/load/run_matrix_pgeo.py (pgeo); accuracy: tests/accuracy/; compatibility: tests/compat/.

Appendix B. pgeo storage

Table 24. pgeo tables (including their indexes).

Table	Size (incl. indexes)
feature	612 MB
admin	22 MB
street_name	16 MB
ac_prefix	3 MB
town	0 MB

Table	Size (incl. indexes)
build_info	0 MB

Table 25. Largest pgeo indexes.

Index	Size
feature_gid_idx	52 MB
feature_geom_idx	36 MB
feature_addr_idx	33 MB
feature_addr_street_hn_idx	23 MB
feature_addr_hn_idx	23 MB
feature_pkey	19 MB
feature_tokens_idx	17 MB
feature_name_trgm_idx	13 MB
feature_layer_idx	6 MB
feature_name_idx	6 MB
street_name_pkey	5 MB
street_name_trgm_idx	4 MB